

## ✓ Lab 5

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
import requests
```

```
image_url=""
https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcRiBWYFazCzI4ZamD6KuYXUuj8qtVaOMmNy4-V8oqhT7Q&s=10
"""
response = requests.get(image_url)
response.raise_for_status()
image_array_1 = np.frombuffer(response.content, np.uint8)
gray_1 = cv2.imdecode(image_array_1, cv2.IMREAD_GRAYSCALE)
```

```
img=img1=-gray_1
```

## ✓ Gaussian Noise

```
mean = 0
std=30
gaussian_noise = np.random.normal(mean, std, img.shape).astype(np.float32)
noisy_image = np.clip(img + gaussian_noise, 0, 255).astype(np.uint8)
noise_mask = cv2.normalize(gaussian_noise, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

plt.figure(figsize=(10, 8))

plt.subplot(2,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(noise_mask, cmap='gray')
plt.title('Gaussian Noise Mask')
plt.axis('off')

plt.subplot(2,2,3)
plt.hist(gaussian_noise.ravel(), bins=100, color='black')
plt.title('Gaussian Noise Histogram')

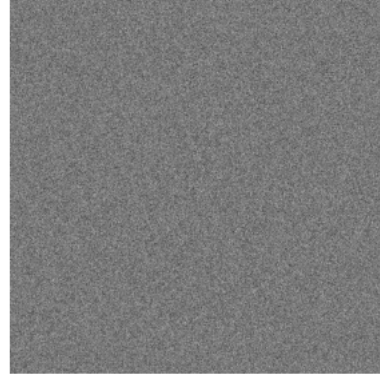
plt.subplot(2,2,4)
plt.imshow(noisy_image, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.show()
```

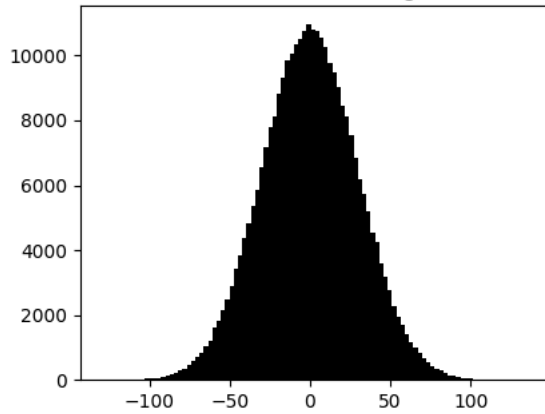
Original Image



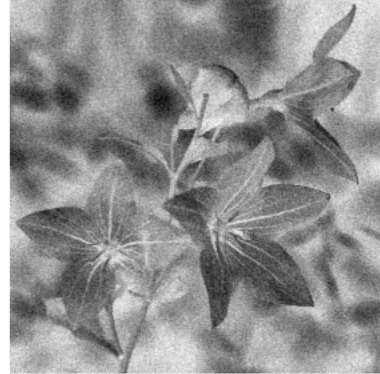
Gaussian Noise Mask



Gaussian Noise Histogram



Noisy Image



## ✓ Rayleigh Noise

```
from numpy.random import rayleigh
scale=80
rayleigh_noise=np.random.rayleigh(scale,size=img.shape)

noise_mask=cv2.normalize(rayleigh_noise.astype(np.float32),None,0,255,cv2.NORM_MINMAX).astype(np.uint8)

noisy_image=np.clip(img.astype(np.float32)+rayleigh_noise,0,255).astype(np.uint8)

plt.subplot(2,2,1)
plt.imshow(img,cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(noise_mask,cmap='gray')
plt.title('Rayleigh Noise Mask ')
plt.axis('off')

plt.subplot(2,2,3)
plt.hist(rayleigh_noise.ravel(),bins=100,color='black')
plt.title('Rayleigh Noise Histogram')

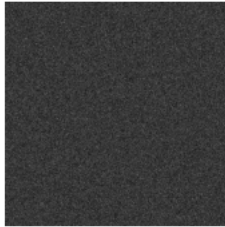
plt.subplot(2,2,4)
plt.imshow(noisy_image,cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.show()
```

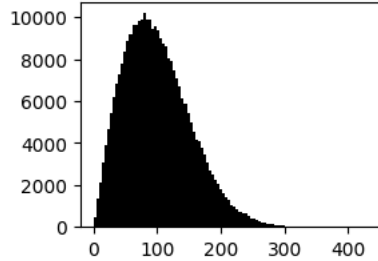
Original Image



Rayleigh Noise Mask



Rayleigh Noise Histogram



Noisy Image



## ✓ Erlang Noise

```
shape = 3.0
scale = 20.0

gamma_noise = np.random.gamma(shape, scale, img.shape).astype(np.float32)
noise_mask = cv2.normalize(gamma_noise, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
noisy_image = np.clip(img.astype(np.float32) + gamma_noise, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noise_mask, cmap='gray')
plt.title('Gamma Noise Mask')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.hist(gamma_noise.ravel(), bins=100, color='black')
plt.title('Gamma Noise Histogram')

plt.subplot(2, 2, 4)
plt.imshow(noisy_image, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.show()
```

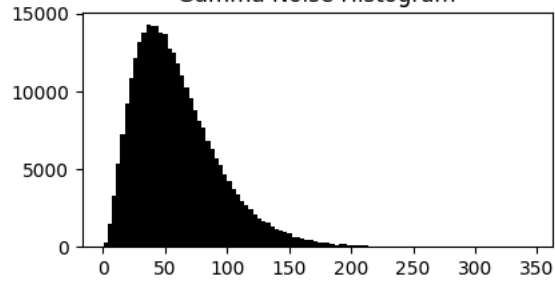
Original Image



Gamma Noise Mask



Gamma Noise Histogram



Noisy Image



## Uniform Noise

```
low = 80
high = 180

uniform_noise = np.random.uniform(low, high, img.shape).astype(np.float32)

noise_mask = cv2.normalize(uniform_noise, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

noisy_image = np.clip(img.astype(np.float32) + uniform_noise, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noise_mask, cmap='gray')
plt.title('Uniform Noise Mask')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.hist(uniform_noise.ravel(), bins=100, color='black')
plt.title('Uniform Noise Histogram')

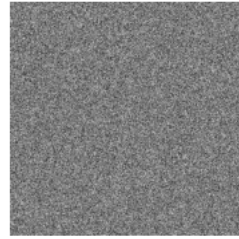
plt.subplot(2, 2, 4)
plt.imshow(noisy_image, cmap='gray')
plt.title('Uniform Noise Image')
plt.axis('off')

plt.show()
```

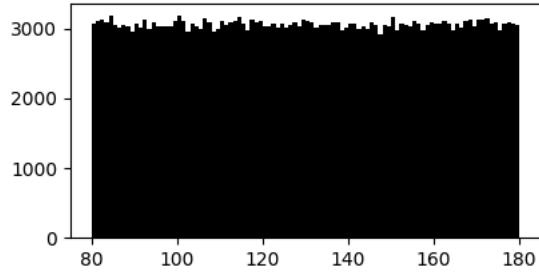
Original Image



Uniform Noise Mask



Uniform Noise Histogram



Uniform Noise Image



## ✓ Salt and Pepper Noise

```
salt_prob = 0.3
pepper_prob = 0.3
rows, cols = img.shape
noisy_image = img.copy()

noise_mask = np.full((rows, cols), 128, dtype=np.uint8)

num_salt = int(salt_prob * rows * cols)
salt_row = np.random.randint(0, rows, num_salt)
salt_col = np.random.randint(0, cols, num_salt)
noisy_image[salt_row, salt_col] = 255
noise_mask[salt_row, salt_col] = 255

num_pepper = int(pepper_prob * rows * cols)
pepper_row = np.random.randint(0, rows, num_pepper)
pepper_col = np.random.randint(0, cols, num_pepper)
noisy_image[pepper_row, pepper_col] = 0
noise_mask[pepper_row, pepper_col] = 0

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noise_mask, cmap='gray')
plt.title('Salt and Pepper Noise Mask')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.hist(noise_mask.ravel(), bins=256, range=(0, 255), color='black')
plt.title('Noise Histogram')
plt.xlabel('Intensity Value')
plt.ylabel('Frequency')

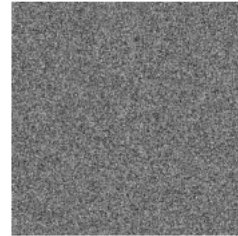
plt.subplot(2, 2, 4)
plt.imshow(noisy_image, cmap='gray')
plt.title('Salt and Pepper Image')
plt.axis('off')

plt.show()
```

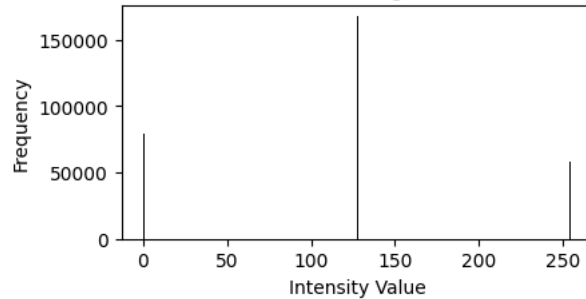
Original Image



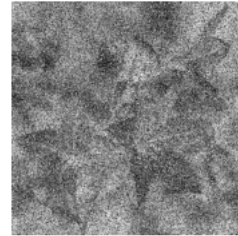
Salt and Pepper Noise Mask



Noise Histogram



Salt and Pepper Image



## Image Restoration

### Arithmetic mean Filter

```
noise=np.random.randint(0,256,size=img.shape,dtype=np.uint8)
noisy_img=np.clip(img.astype(np.int16)+noise,0,255).astype(np.uint8)

kernel_size=3
pad=kernel_size//2

padded=np.pad(noisy_img,pad,mode= 'constant')

filtered = np.zeros_like(noisy_img, dtype=np.uint8)

for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window=padded[i:i+kernel_size,j:j+kernel_size]
        filtered[i,j]=np.mean(window)
plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Filtered Image')
plt.axis('off')

plt.show()
```

Original Image



Noisy Image



Filtered Image



## ✓ Geometric Mean Filter

```
noise=np.random.randint(0,256,size=img.shape,dtype=np.uint8)
noisy_img=np.clip(img.astype(np.int16)+noise,0,255).astype(np.uint8)

kernel_size=3
pad=kernel_size//2

padded=np.pad(noisy_img,pad,mode= 'constant')

filtered = np.zeros_like(noisy_img, dtype=np.uint8)

for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window=padded[i:i+kernel_size,j:j+kernel_size]
        window=np.where(window==0,1,window)

        filtered[i,j]=np.exp(np.mean(np.log(window)))

filtered=np.clip(filtered,0,255).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Filtered Image')
plt.axis('off')

plt.show()
```

Original Image



Noisy Image



Filtered Image



## ✓ Harmonic mean

```
noise=np.random.randint(0,256,size=img.shape,dtype=np.uint8)
noisy_img=np.clip(img.astype(np.int16)+noise,0,255).astype(np.uint8)

kernel_size=3
pad=kernel_size//2

padded=np.pad(noisy_img,pad,mode= 'constant')

filtered = np.zeros_like(noisy_img, dtype=np.uint8)

for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window=padded[i:i+kernel_size,j:j+kernel_size]
        window=np.where(window==0,1>window)

        filtered[i,j]=(kernel_size*kernel_size)/np.sum(1/window)

filtered=np.clip(filtered,0,255).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Harmonic Mean Filtered Image')
plt.axis('off')

plt.show()
```



Original Image



Noisy Image



Harmonic Mean Filtered Image



## ✓ Contra Harmonic Mean

```
noisy_img = img.copy()

noise_density = 0.05
num_salt = int(noise_density * img.size / 2)
num_pepper = int(noise_density * img.size / 2)

#salt noise
salt_coords = np.random.randint(0, img.shape[0], num_salt), np.random.randint(0, img.shape[1], num_salt)
noisy_img[salt_coords] = 255

#pepper noise
pepper_coords = np.random.randint(0, img.shape[0], num_pepper), np.random.randint(0, img.shape[1], num_pepper)
noisy_img[pepper_coords] = 0

#contra Harmonic mean
kernel_size = 3
Q = -2          # Q < 0 removes salt noise
pad = kernel_size // 2
epsilon = 1e-10 # Tiny value to prevent division by zero

noisy_img_float = noisy_img.astype(np.float32) + epsilon

padded = np.pad(noisy_img_float, pad, mode='constant', constant_values=epsilon)

filtered = np.zeros_like(noisy_img, dtype=np.float32)

for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window = padded[i:i + kernel_size, j:j + kernel_size]

        numerator = np.sum(np.power(window, Q + 1))
        denominator = np.sum(np.power(window, Q))

        filtered[i, j] = numerator / denominator

filtered = np.clip(filtered, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 8))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')
```

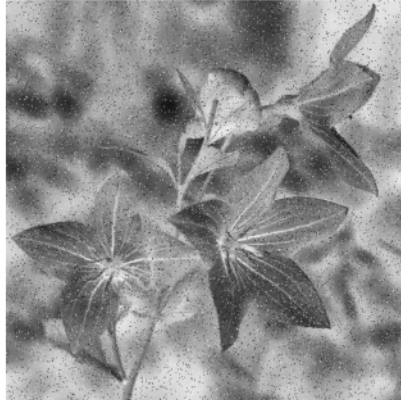
```
plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('contra Harmonic Filtered Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

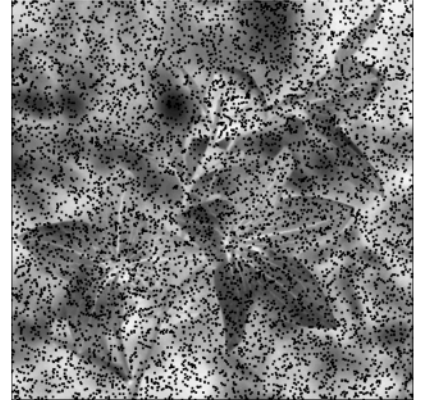
Original Image



Noisy Image



contra Harmonic Filtered Image



## ✦ Remaining to Do

## ✦ Minimum Filter

```
noisy_img = img.copy()

noise_density = 0.05
num_salt = int(noise_density * img.size / 2)
num_pepper = int(noise_density * img.size / 2)

#salt noise
salt_coords = np.random.randint(0, img.shape[0], num_salt), np.random.randint(0, img.shape[1], num_salt)
noisy_img[salt_coords] = 255

#pepper noise
pepper_coords = np.random.randint(0, img.shape[0], num_pepper), np.random.randint(0, img.shape[1], num_pepper)
noisy_img[pepper_coords] = 0

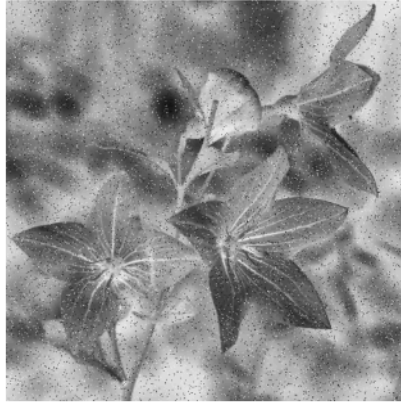
#contra Harmonic mean
kernel_size = 3
Q = -2 # Q < 0 removes salt noise
pad = kernel_size // 2
noisy_img_float = noisy_img.astype(np.float32)
padded = np.pad(noisy_img_float, pad, mode='constant', constant_values=epsilon)
filtered = np.zeros_like(noisy_img, dtype=np.float32)
for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window = padded[i:i + kernel_size, j:j + kernel_size]

        filtered[i, j] = np.min(window)
filtered = np.clip(filtered, 0, 255).astype(np.uint8)
plt.figure(figsize=(10, 8))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Minimum Filtered Image')
plt.axis('off')
plt.tight_layout()
plt.show()
```

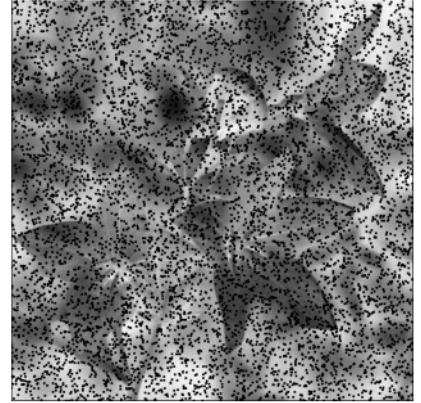
Original Image



Noisy Image



Minimum Filtered Image



## Maximum Filter

```
noisy_img = img.copy()
noise_density = 0.05
num_salt = int(noise_density * img.size / 2)
num_pepper = int(noise_density * img.size / 2)

#salt noise
salt_coords = np.random.randint(0, img.shape[0], num_salt), np.random.randint(0, img.shape[1], num_salt)
noisy_img[salt_coords] = 255

#pepper noise
pepper_coords = np.random.randint(0, img.shape[0], num_pepper), np.random.randint(0, img.shape[1], num_pepper)
noisy_img[pepper_coords] = 0

#contra Harmonic mean
kernel_size = 3
Q = -2 # Q < 0 removes salt noise
pad = kernel_size // 2
noisy_img_float = noisy_img.astype(np.float32)
padded = np.pad(noisy_img_float, pad, mode='constant', constant_values=epsilon)
filtered = np.zeros_like(noisy_img, dtype=np.float32)
for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window = padded[i:i + kernel_size, j:j + kernel_size]

        filtered[i, j] = np.max(window)
filtered = np.clip(filtered, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 8))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

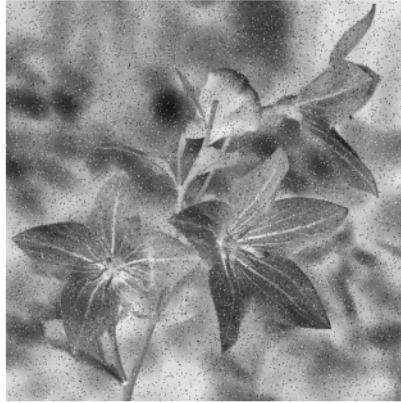
plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Maximum Filtered Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original Image



Noisy Image



Maximum Filtered Image



## Median Filter

```
noisy_img = img.copy()
noise_density = 0.05
num_salt = int(noise_density * img.size / 2)
num_pepper = int(noise_density * img.size / 2)

#salt noise
salt_coords = np.random.randint(0, img.shape[0], num_salt), np.random.randint(0, img.shape[1], num_salt)
noisy_img[salt_coords] = 255

#pepper noise
pepper_coords = np.random.randint(0, img.shape[0], num_pepper), np.random.randint(0, img.shape[1], num_pepper)
noisy_img[pepper_coords] = 0

#contra Harmonic mean
kernel_size = 3
Q = -2 # Q < 0 removes salt noise
pad = kernel_size // 2

noisy_img_float = noisy_img.astype(np.float32)
padded = np.pad(noisy_img_float, pad, mode='constant', constant_values=epsilon)
filtered = np.zeros_like(noisy_img, dtype=np.float32)
for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window = padded[i:i + kernel_size, j:j + kernel_size]

        filtered[i,j] = np.median(window)

filtered = np.clip(filtered, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 8))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Median Filtered Image')
plt.axis('off')

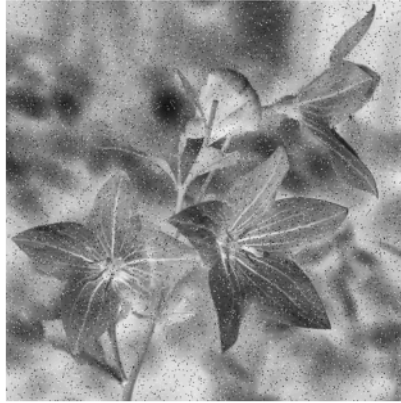
plt.tight_layout()
plt.show()
```



Original Image



Noisy Image



Median Filtered Image



## Mid point filter

```
noise=np.random.uniform(low=-30, high=30, size=img.shape)
noisy_img=np.clip(img.astype(np.float32)+noise,0,255).astype(np.uint8)

kernel_size = 3
pad = kernel_size // 2
noisy_img_float = noisy_img.astype(np.float32)
padded = np.pad(noisy_img_float, pad, mode='constant')
filtered = np.zeros_like(noisy_img, dtype=np.float32)
for i in range(noisy_img.shape[0]):
    for j in range(noisy_img.shape[1]):
        window = padded[i:i + kernel_size, j:j + kernel_size]
        min_value=np.min(window)
        max_value=np.max(window)

        filtered[i,j] = (min_value+max_value)/2

filtered = np.clip(filtered, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Midpoint Filtered Image') # Corrected title
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original Image



Noisy Image



Midpoint Filtered Image



## ✓ Adaptive Filter

```
noisy_img = img.copy()

noise_density = 0.05
num_salt = int(noise_density * img.size / 2)
num_pepper = int(noise_density * img.size / 2)

# Salt noise
salt_coords = np.random.randint(
    0, img.shape[0], num_salt), np.random.randint(0, img.shape[1], num_salt)
noisy_img[salt_coords] = 255

# Pepper noise
pepper_coords = np.random.randint(
    0, img.shape[0], num_pepper), np.random.randint(0, img.shape[1], num_pepper)
noisy_img[pepper_coords] = 0

def adaptive_median_filter(img, max_window=7):
    rows, cols = img.shape
    output = img.copy()
    for i in range(rows):
        for j in range(cols):
            window_size = 3
            while window_size <= max_window:
                pad = window_size // 2
                r1 = max(i - pad, 0)
                r2 = min(i + pad + 1, rows)
                c1 = max(j - pad, 0)
                c2 = min(j + pad + 1, cols)

                window = img[r1:r2, c1:c2]
                z_min = np.min(window)
                z_max = np.max(window)
                z_median = np.median(window)
                z_xy = img[i, j]

                # Stage A
                if z_min < z_median < z_max:
                    # Stage B
                    if z_min < z_xy < z_max:
                        output[i, j] = z_xy
                    else:
                        output[i, j] = z_median
                    break
                else:
                    window_size += 2
            if window_size > max_window:
                output[i, j] = z_median
    return output

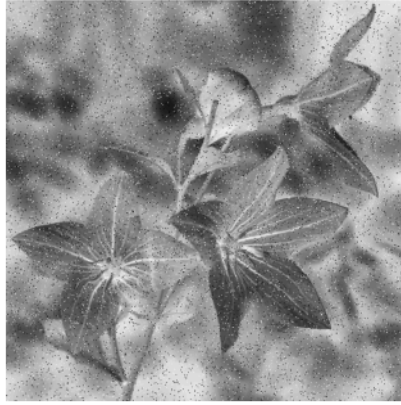
filtered = adaptive_median_filter(noisy_img, max_window=7)
filtered = np.clip(filtered, 0, 255).astype(np.uint8)
plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(noisy_img, cmap='gray')
plt.title('Noisy Image')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap='gray')
plt.title('Adaptive Filtered Image')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Original Image



Noisy Image



Adaptive Filtered Image

